FRC JAVA CODING LAB 7.0

OBSERVATION MODEL CONTRACT

Immutable mechanism meaning derived from approved inputs

Document	Version	Status	Language
OC-01	1.0	FROZEN	English

SSIS FRC Team 10951
Author: SSIS | Mentor: SSIS

1. Contract

An Observation is an immutable, vendor-neutral description of mechanism or estimator state at a defined time. It is created by the subsystem or estimator after reading IOInputs. It is safe to hand to telemetry, diagnostics, tests, and logging without exposing mutable hardware transport state.

2. IOInputs vs Observation

IOInputs is mutable transport data populated by IO once per periodic cycle. It mirrors hardware facts and may include connection or configuration fields. Observation is immutable domain meaning selected or derived by a subsystem or estimator.

Do not publish or retain a mutable IOInputs object as the public observation contract. Copy required values into the Observation.

3. Observation vs Telemetry

Observation defines data and meaning. Telemetry defines destinations, topics, rates, serialization, and publication. An Observation has no NetworkTables keys, publishers, log writers, dashboards, or update loops. Telemetry does not calculate mechanism behavior and must not mutate the Observation.

4. Required Model Properties

Immutable after construction; complete for its declared purpose; vendor-neutral; deterministic for the same source values; explicit units; explicit timing basis when time matters; explicit validity when a value may be absent, stale, disconnected, estimated, or unsupported.

Prefer Java records or final classes with final fields and defensive copies. Collections, arrays, poses, and nested values must not expose mutable aliases.

5. Naming and Units

Use <Mechanism>Observation and, when needed, <Mechanism>ObservationEvaluator. Use PascalCase types and camelCase components. Put units in names: velocityRpm, positionRotations, supplyCurrentAmps, temperatureCelsius, timestampSeconds. Normalized outputs must say appliedOutput or dutyCycle and document the range.

Enums describe observation categories such as STOPPED, INTAKING, or DISCONNECTED; they do not command behavior. Avoid vague names such as value, data, status, speed, or time.

6. Timing and Validity

Every Observation represents one coherent logical sample. If fields originate at different device times, document the aggregation rule. Use the project-approved monotonic robot time base for timestampSeconds; never use wall-clock time for control-period observations.

When freshness matters, include timestampSeconds and a validity representation such as valid, connected, ageSeconds, or a small status enum. Invalid numeric data must not silently masquerade as a real zero. Unsupported values must be explicit and deterministic.

7. Equality, Serialization, and Evolution

Value equality is encouraged. Serialization belongs outside observation unless it is a vendor-neutral, side-effect-free representation approved for logging. Additive fields require review of constructors, tests, telemetry, and logs. Renames, unit changes, removals, and semantic changes are breaking changes.

8. Anti-Patterns

Mutable public fields; retaining IOInputs by reference; vendor types; NetworkTables topic names; getters that read hardware; commands embedded in enums; scheduler calls; RobotContainer access; hidden time reads; nullable values without a contract; telemetry-only duplicate models.

9. Acceptance Criteria

The model can be constructed in a unit test without WPILib runtime or hardware. Repeated reads do not change it. Telemetry can consume it without knowing IO or vendors. Units, time, validity, and ownership are reviewable from the type and JavaDoc.

Revision History

1.0 | 2026-08-01 | FROZEN | Initial release